

ARGO CD

PRODUCTION HANDBOOK



FROM GITOPS FUNDAMENTALS TO ENTERPRISE PRODUCTION OPERATIONS

GIT IS
THE SOURCE
OF TRUTH



WHAT IS ARGO CD?

A declarative, GitOps continuous delivery tool for Kubernetes. It ensures your applications are always in the desired state as defined in Git.



CACHE / STATE

CORE PRINCIPLES

- ✓ DECLARATIVE
- ✓ GIT AS SOURCE OF TRUTH
- ✓ AUTOMATED & CONTINUOUS
- ✓ SECURE BY DESIGN
- ✓ AUDITABLE & REPEATABLE

WHAT YOU WILL LEARN

- GitOps fundamentals
- Argo CD architecture
- Applications & Projects
- Sync, Hooks & Waves
- AppOfApps & ApplicationSet
- Multi-cluster GitOps
- Security, RBAC & SSO
- Observability & Troubleshooting
- Production best practices



PRODUCTION
READY
GITOPS!

PRODUCTION NOTE

Argo CD is not just a deployment tool. It is part of your production control plane. Treat it with the same care and security as any critical infrastructure system.



AUTOMATE EVERYTHING. TRUST GIT. DELIVER CONFIDENTLY.

1. GITOPS AND ARGO CD ARCHITECTURE

CORE CONCEPTS

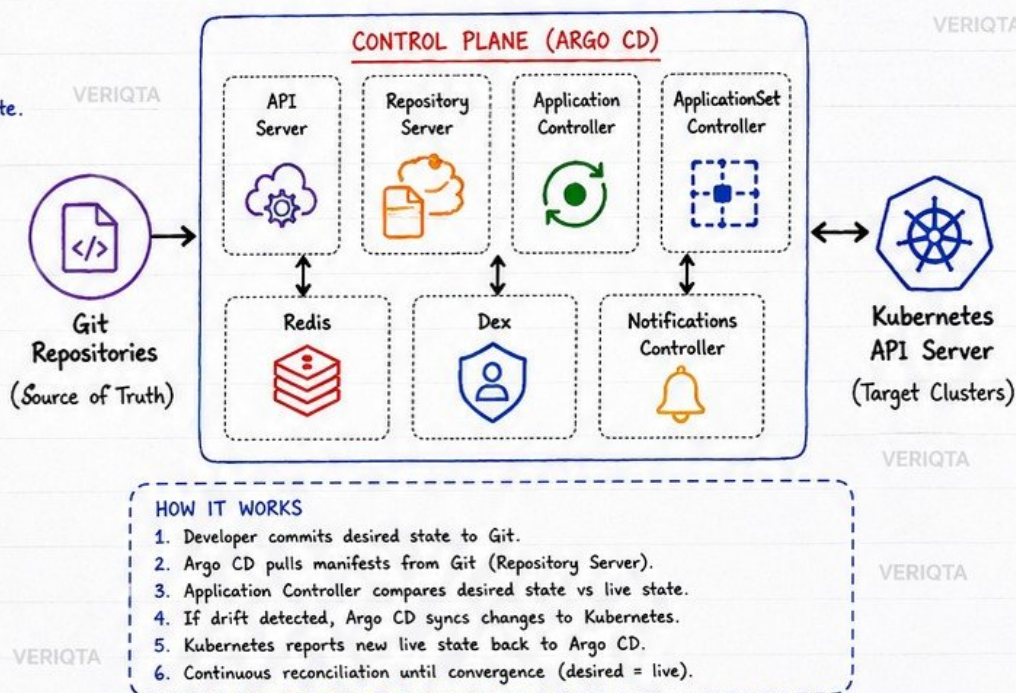
- GitOps operating model
- Desired state vs live state
- Git as the source of truth
- Pull-based continuous delivery
- Argo CD control plane
- Kubernetes API interaction
- Declarative application delivery
- Reconciliation and convergence
- Argo CD vs traditional push-based CI/CD

PRODUCTION FOCUS

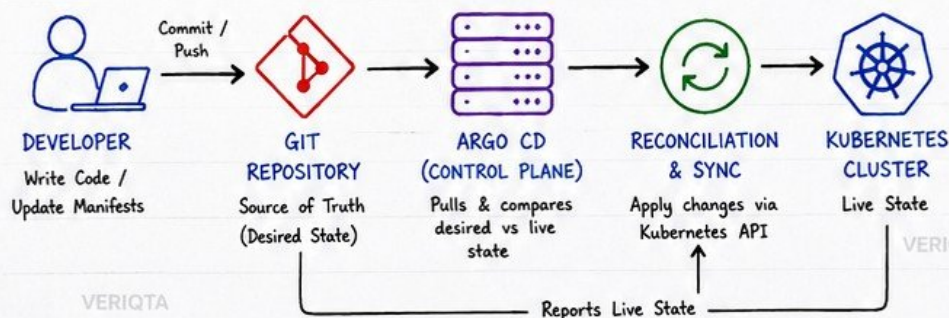
- ✓ CI vs CD responsibility boundaries
CI builds, tests & produces artifacts.
CD (Argo CD) delivers declarative state.
- ✓ Why CI should not directly modify production clusters
Direct writes bypass auditability, review, policy, and rollback safety.
- ✓ GitOps trust boundaries
Git repo = trust root.
Argo CD pulls.
Clusters never push configuration.
- ✓ Control-plane failure implications
Argo CD outage stops delivery visibility and sync, but workloads keep running.
Design for HA and disaster recovery.

ARCHITECTURE COMPONENTS

- API Server: Exposes REST/gRPC API, UI and CLI access
- Repository Server: Fetches & caches Git repos, generates manifests
- Application Controller: Watches apps, compares state, reconciles & syncs
- Redis: Caching, state storage, rate limiting, locks, app status
- Dex: SSO / OIDC authentication provider
- Notifications Controller: Triggers & sends notifications
- ApplicationSet Controller: Generates Applications from templates (clusters, repos, etc.)
- Kubernetes API Server: Target clusters' control plane
- Git Repositories: Single source of truth (manifests, apps, configs, Kustomize, Helm, etc.)



GITOPS RECONCILIATION FLOW



GITOPS vs TRADITIONAL CI/CD

GITOPS (ARGO CD)	TRADITIONAL PUSH CI/CD
↓ Pull model (Argo CD pulls)	↑ Push model (CI pushes)
Git is Source of Truth	CI system is Source of Truth
Declarative state	Imperative commands
Continuous reconciliation	One-time deployment
Audit, Review, Rollback via Git	Hard to audit / rollback

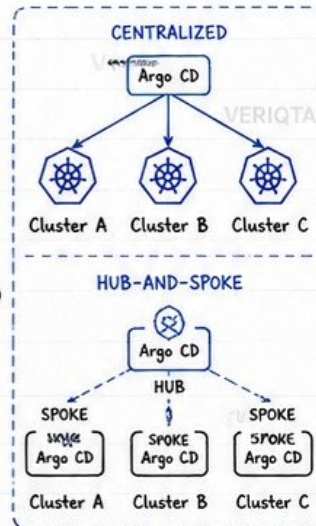
PRODUCTION NOTE

Argo CD should continuously reconcile declared state. It should not become another imperative deployment script runner. Keep intent in Git. Let Argo CD ensure convergence.

2. INSTALLING AND DESIGNING ARGO CD FOR PRODUCTION

1. DEPLOYMENT MODELS

- Standard installation
- Core installation
- Namespace-scoped installation
- Cluster-wide installation
- High-availability deployment
- Centralized vs per-cluster Argo CD
- Hub-and-spoke architecture



2. PRODUCTION DESIGN

- | | |
|-----------------------------------|---|
| ✓ Dedicated Namespace | e.g. argocd |
| ✓ HA Controllers | Run multiple replicas with leader election |
| ✓ Repository Server Scaling | Scale based on repo count and generation load |
| ✓ Redis HA Considerations | Use HA Redis with persistence (AWS ElastiCache, Redis Sentinel, etc.) |
| ✓ Ingress Exposure | Expose via Ingress or API Gateway |
| ✓ TLS Termination | Terminate TLS at Ingress / Gateway |
| ✓ Persistent vs Ephemeral | Redis, Repo-Server cache can be persistent; controllers are ephemeral |
| ✓ Resource Requests & Limits | Set proper CPU / Memory |
| ✓ PodDisruptionBudgets | Ensure availability during upgrades |
| ✓ Anti-Affinity & Topology Spread | Distribute pods across nodes / AZs |

3. OPERATIONS



CLI INSTALLATION

- Download Argo CD CLI
- argocd version
- Login to the server
argocd login <ARGOCD_SERVER>



INITIAL AUTHENTICATION

- Default admin user on first install: admin / <password from secret>
- Change immediately



ADMIN ACCOUNT HANDLING

- Change default password
- Create SSO / OIDC integration
- Disable local admin after SSO



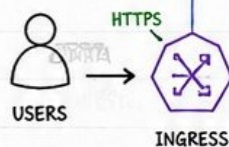
VERSION VERIFICATION

- argocd version --client
- --server

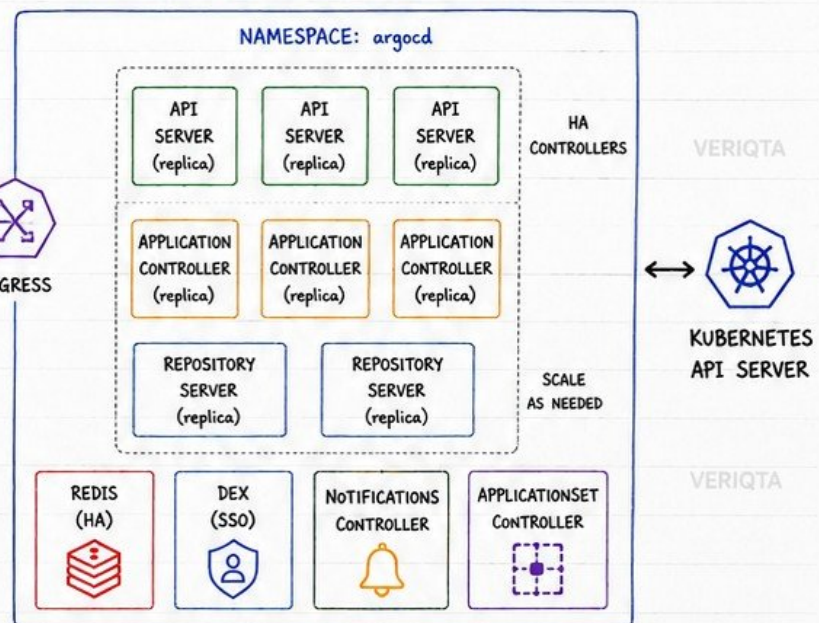


UPGRADE PLANNING

- Check release notes
- Verify compatibility (K8s, CRDs)
- Backup configuration
- Test in staging environment
- Plan rollback strategy



TYPICAL PRODUCTION DEPLOYMENT (HA)



PRODUCTION CHECKLIST

- ✓ Dedicated namespace
- ✓ HA enabled
- ✓ Proper resource limits
- ✓ PDB configured
- ✓ TLS enabled
- ✓ Backups & recovery
- ✓ Monitoring & alerts

★ SENIOR ENGINEER TIP

Choose the Argo CD topology based on blast radius and organizational boundaries, not convenience.

3. APPLICATIONS AND THE ARGO CD APPLICATION CRD

1. APPLICATION ANATOMY

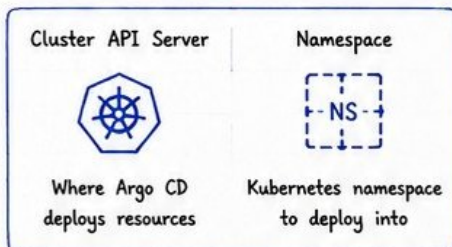
- apiVersion → API group & version
- kind → Resource type (Application)
- metadata → Name, labels, annotations
- spec.project → AppProject the app belongs to
- spec.source → Where manifests come from
- spec.destination → Where to deploy
- spec.syncPolicy → How Argo CD syncs



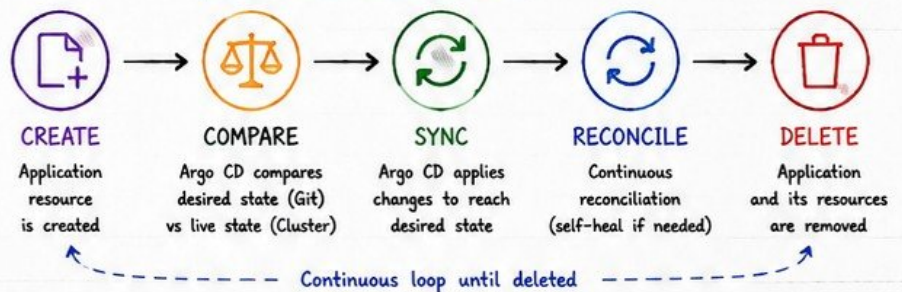
2. SOURCE CONFIGURATION

SOURCE TYPE	DESCRIPTION
Git repository	Fetch manifests from Git (most common)
Target revision	Branch, tag, or commit SHA
Path	Path to manifests inside the repo
Helm source	Chart from repo, with values
Kustomize source	Kustomize base/overlay
Directory source	Plain directory with YAML/JSON
Multiple sources	Combine multiple sources in one Application

3. DESTINATION CONFIGURATION



4. APPLICATION LIFECYCLE



5. HANDS-ON: DECLARATIVE APPLICATION MANIFEST

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: guestbook
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/argoproj/argocd-example-apps.git
    targetRevision: HEAD
    path: guestbook
  destination:
    server: https://kubernetes.default.svc
    namespace: guestbook
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

This manifest is tracked in Git. Argo CD reads it and ensures the cluster matches this desired state.

6. HANDS-ON: CLI-CREATED APPLICATION

```
argocd app create guestbook \
  --repo https://github.com/argoproj/
  argocd-example-apps.git \
  --path guestbook \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace guestbook \
  --sync-policy automated --self-heal --prune
```



Note:
CLI creates the same Application CRD behind the scenes. Prefer Git-based declaration for production.

7. HANDS-ON: APPLICATION STATUS INSPECTION

```
# Get application details
argocd app get guestbook
# Check sync status
argocd app status guestbook
# Show differences
argocd app diff guestbook
# View resources
argocd app resources guestbook
# View manifests rendered
argocd app manifests guestbook
# View history
argocd app history guestbook
```

SYNC STATUS

- Synced
- OutOfSync
- Syncing
- Unknown

HEALTH STATUS

- ♥ Healthy
- ♥ Progressing
- ♥ Degraded
- ♥ Missing



PRODUCTION NOTE

Treat Application manifests as infrastructure code. Do not rely on UI-created configuration for production environments.



4. RECONCILIATION, SYNC, AND APPLICATION STATE

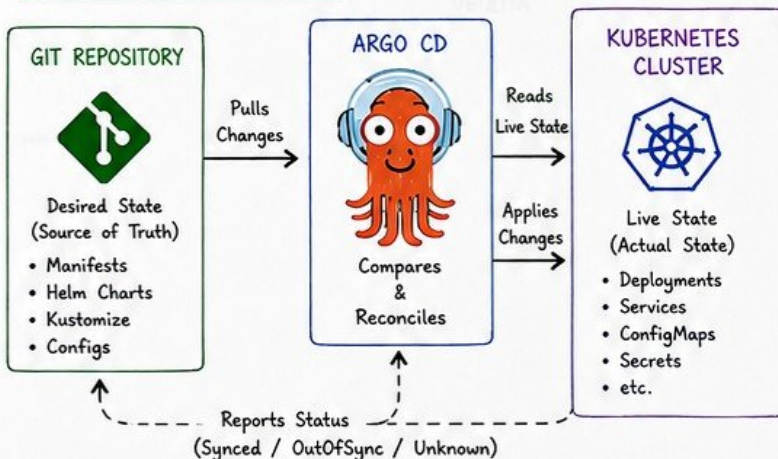
1. UNDERSTAND THE ARGO CD ENGINE

	Desired State (Git)	The state defined in Git (manifests, configs, charts).
	Live State (Cluster)	The current state of resources in the Kubernetes cluster.
	Comparison	Argo CD compares desired vs live state to detect differences.
	Reconciliation Loop	Argo CD continuously works to make live state match desired state.
	Synced	Desired state matches live state. No differences.
	OutOfSync	There are differences between desired and live state.
	Unknown	Argo CD cannot determine the resource state.
	Resource Health	Health of individual Kubernetes resources.
	Application Health	Overall health of the application based on its resources.

2. SYNC OPERATIONS

OPERATION		DESCRIPTION	WHEN TO USE
	Manual Sync	User triggers sync from UI, CLI, or API.	One-time deployments, testing, controlled releases.
	Automated Sync	Argo CD automatically syncs when changes are detected.	Continuous delivery, GitOps best practice.
	Selective Sync	Sync specific resources or resource groups.	Partial updates, advanced use cases.
	Pruning	Remove resources from the cluster that are no longer in Git.	Keep cluster clean and in sync with Git.
	Self-Healing	Automatically revert drift caused by manual changes.	Prevent configuration drift.
	Retry Behavior	Automatically retry failed syncs based on policy.	Handle transient failures, improve reliability.

4. RECONCILIATION DIAGRAM



3. PRODUCTION SCENARIOS

	Manual Cluster Modification Someone changes a resource in the cluster outside of Git. Argo CD detects drift.
	Git Change A commit is pushed to Git. Argo CD pulls the change and syncs it.
	Resource Deletion A resource is deleted in the cluster. Argo CD recreates it (self-heal).
	Failed Reconciliation Sync fails due to validation, permission, or runtime errors. Argo CD retries.
	Configuration Drift Cluster state differs from Git over time. Argo CD detects and fixes it.

5. APPLICATION STATE OVERVIEW

STATE	DESCRIPTION		INDICATOR
	Synced	Desired state matches live state. Everything is in sync.	Green Check
	OutOfSync	Differences exist between desired and live state.	Orange Dot
	Unknown	Argo CD cannot determine the current state of the application or resource.	Gray Question
	Healthy	All resources are healthy.	Green Heart
	Degraded	One or more resources are not healthy.	Orange Heart
	Missing	Resource is missing or deleted.	Red Heart



DEBUGGING INSIGHT

An OutOfSync application is not automatically broken. It simply means the live state differs from the desired state.

Before forcing a sync, always ask:

- ☒ Which field is different?
- ☒ Who owns that field (Git or Cluster)?
- ☒ Was the change intentional?
- ☒ Will syncing cause downtime or data loss?
- ☒ Is this a temporary or permanent change?

Investigate first. Then decide.

5. SYNC POLICIES, SYNC OPTIONS, AND RESOURCE MANAGEMENT

1. AUTOMATED SYNCHRONIZATION

OPTION	DESCRIPTION
 automated	Enables automated synchronization. Argo CD syncs changes automatically when detected.
 prune	Automatically delete resources from the cluster that are no longer defined in Git.
 selfHeal	Automatically revert out-of-band changes in the cluster to match the declared state.
 allowEmpty	Allows automated sync even if the application has no resources defined (e.g., empty directory).



Example syncPolicy (automated):

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
    allowEmpty: false
```

2. IMPORTANT SYNC OPTIONS

OPTION	PURPOSE	WHEN TO USE
CreateNamespace	Create the destination namespace if it does not exist.	When deploying into new namespaces.
PruneLast	Prune resources only after all other changes are applied.	Avoid deleting resources before new ones are ready.
PrunePropagationPolicy	Set deletion propagation policy (Foreground Background Orphan).	Control how Kubernetes deletes dependent resources.
ApplyOutOfSyncOnly	Apply changes only to resources that are OutOfSync.	Improve performance on large applications.
ServerSideApply	Use Kubernetes Server-Side Apply instead of client-side apply.	Better field ownership and conflict management.
Replace	Use kubectl replace instead of apply.	When apply fails due to immutable field changes.
Force	Force resource update even when server-side apply conflicts.	Break through conflicts (last resort).
FailOnSharedResource	Fail sync if resource is managed by another Application.	Prevent accidental overwrite of resources.
RespectIgnoreDifferences	Respect ignoreDifferences rules when detecting drift.	Avoid syncing fields that should be ignored.



Example syncOptions:

```
syncPolicy:
  syncOptions:
    - CreateNamespace=true
    - PruneLast=true
    - ApplyOutOfSyncOnly=true
    - ServerSideApply=true
    - RespectIgnoreDifferences=true
```

3. RESOURCE BEHAVIOR

	DELETION PROPAGATION	Controls how Kubernetes deletes dependent resources when a resource is removed. • Foreground: Delete dependents first (default) • Background: Delete in background • Orphan: Leave dependents orphaned
	ORPHANED RESOURCES	Resources removed from Git but not deleted from cluster because pruning is disabled or misconfigured. • Can cause cost, security, and compliance issues. • Use pruning or orphaned resource monitoring.
	SHARED-RESOURCE RISKS	Multiple Applications managing the same resource can lead to conflicts and unpredictable behavior. • Use FailOnSharedResource to fail fast. • Enforce clear ownership of resources.
	NAMESPACE METADATA	Namespace labels, annotations, and finalizers can cause drift. • Ensure Argo CD owns the namespace metadata. • Use CreateNamespace and manage metadata in Git.

RESOURCE OWNERSHIP EXAMPLE

APP A

Deployment
frontend

APP B

Deployment
frontend



**CONFLICT
(SHARED
RESOURCE)**

BEST PRACTICE

One Application should own one resource.



4. PRODUCTION FAILURE



Blindly enabling automated pruning can turn a Git mistake into immediate infrastructure deletion.

REAL-WORLD SCENARIO

- Developer accidentally removes a manifest from Git.
- Argo CD automated sync + prune deletes the resource.
- Production service goes down instantly.
- Recovery requires manual intervention and time.



6. SYNC WAVES, HOOKS, AND DEPLOYMENT ORDERING

1. RESOURCE ORDERING

	PHASES	Define the high-level stages of a deployment lifecycle using Hooks.
	WAVES	Numeric order within a phase. Lower waves sync first (-2, -1, 0, 1, 2...). Default wave is 0.
	RESOURCE KINDS	Within the same wave, Argo CD orders by Kubernetes kind (e.g., Namespace → CRD → Deployment → Pod).
	RESOURCE NAMES	If still tied, alphabetical order by resource name is used.

VERIQTA

3. HOOK LIFECYCLE

VERIQTA

	HOOK CREATION	When a sync starts, Argo CD creates hook resources based on the phase.
	HOOK EXECUTION	Hooks are executed as Kubernetes jobs or resources in the defined order.
	SUCCESSFUL HOOK	If the hook succeeds, Argo CD applies the deletion policy.
	FAILED HOOK	If the hook fails, it is retained based on the deletion policy for investigation.
	CLEANUP	Hooks are cleaned up according to the policy to avoid clutter.

HOOK DELETION POLICIES

- HookSucceeded: Delete hook after successful completion.
- HookFailed: Delete hook if it fails.
- HookSucceeded,HookFailed: Delete in both cases.
- BeforeHookCreation: Delete existing hook before creating a new one.
- None: Never delete automatically.

2. HOOKS

VERIQTA

HOOK TYPE	WHEN IT RUNS	PURPOSE	SUCCESS DEFAULT
 PreSync	Before applying resources in Sync phase.	Run tasks before deployment (e.g., DB migration).	Delete
 Sync	During the sync process (in order with waves).	Run tasks in the middle of deployment if needed.	Delete
 PostSync	After all resources are synced successfully.	Run verification or notifications after deployment.	Delete
 SyncFail	When sync fails at any point.	Run diagnostics, alerts, or cleanup on failure.	Hook Succeeded
 PostDelete	After an Application is deleted.	Cleanup external resources created by the app.	Delete
 Skip	Hook is ignored entirely.	Disable a hook without deleting its definition.	N/A

4. PRODUCTION USE CASES

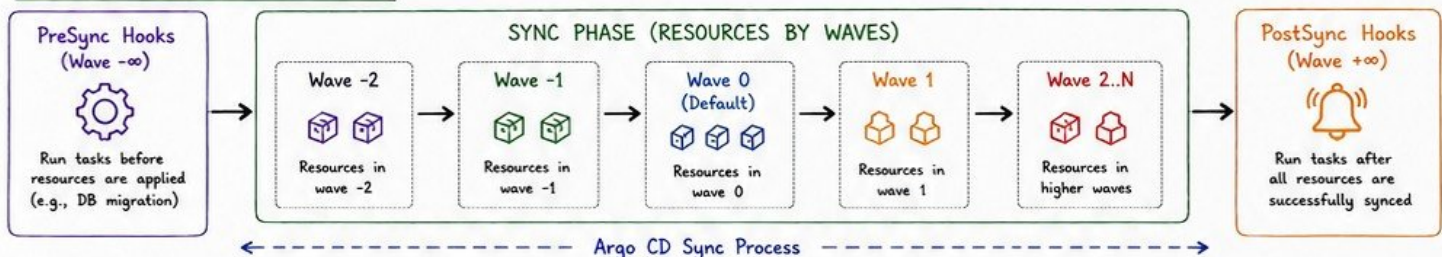
VERIQTA

	DATABASE MIGRATION	Use PreSync hooks to run DB migrations before application resources are deployed.
	SCHEMA VALIDATION	Validate database schema, APIs, or configs before releasing changes.
	SMOKE TESTS	Run quick functional tests after deployment to ensure the release is healthy.
	PRE-DEPLOYMENT CHECKS	Verify dependencies, external services, or feature flags before applying resources.
	POST-DEPLOYMENT VERIFICATION	Run deeper validations, notifications, or register the release in external systems.

5. DEPLOYMENT ORDERING FLOW

VERIQTA

VERIQTA



SENIOR ENGINEER TIP

Use hooks for deployment lifecycle operations.
Do not turn Argo CD into a general-purpose workflow engine.

WHY?

- Hooks should be idempotent and fast.
- Complex workflows belong in dedicated systems.
- Keep deployments predictable and debuggable.



7. REPOSITORY DESIGN AND GITOPS ENVIRONMENT STRATEGY

1. REPOSITORY MODELS

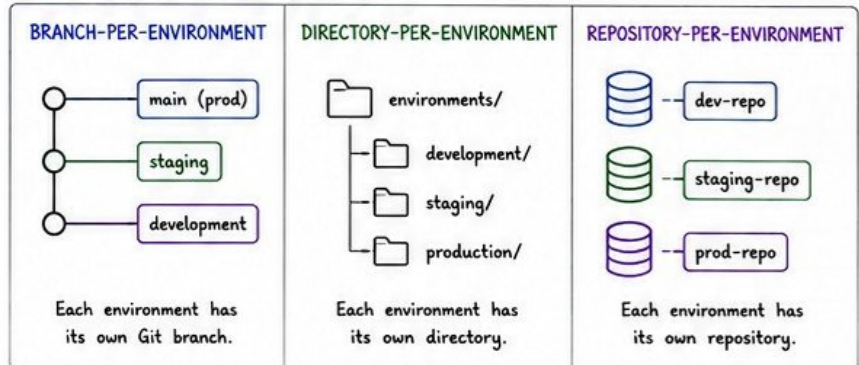
VERIQTA

	MONOREPO	All applications, configurations, and environments live in a single repository.
	POLYREPO	Each application or component has its own repository.
	APPLICATION REPOSITORY	Contains application source code and deployment manifests for that application.
	CONFIGURATION REPOSITORY	Holds environment configs, policies, and shared infrastructure code.
	PLATFORM REPOSITORY	Stores platform components, operators, tools, and base infrastructure.

VERIQTA

2. ENVIRONMENT ORGANIZATION

VERIQTA



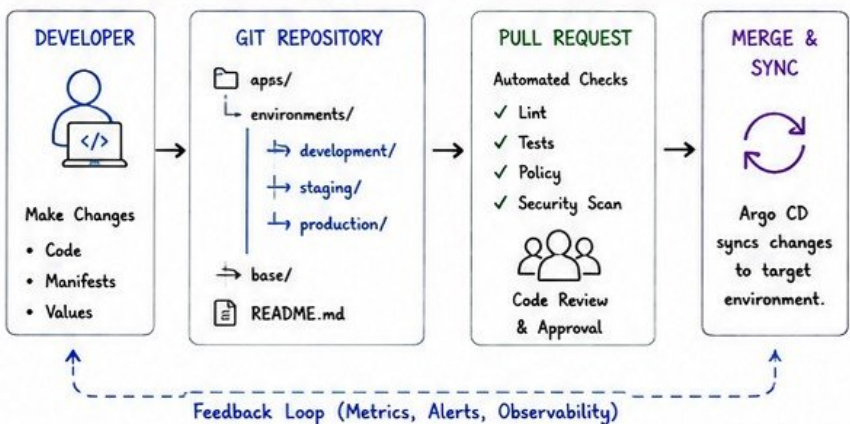
STRATEGY	PROS	CONS	BEST FOR
Branch-per-env	Simple, fast to setup, good for small teams	Branch sprawl, merge conflicts	Small teams, few environments
Directory-per-env	Clear separation, scales well	More complex paths, access control	Most organizations, recommended
Repo-per-env	Strong isolation, fine-grained access	High maintenance, duplication	Strict compliance, separate ownership

3. PROMOTION MODELS

	GIT COMMIT PROMOTION	Same code is promoted across environments by merging or tagging commits.
	PULL REQUESTS	Changes flow between environments using PRs with code review and automated checks.
	IMAGE TAG UPDATES	Promote by updating container image tags in manifests or values files.
	ENVIRONMENT OVERLAYS	Base manifests are overlaid with environment-specific values or patches.

4. EXAMPLE: DIRECTORY-PER-ENVIRONMENT WORKFLOW

VERIQTA



VERIQTA

VERIQTA

5. GOVERNANCE

	PROTECTED BRANCHES Prevent direct pushes to main/production branches.		CODEOWNERS Define code owners for directories and require their review.		REQUIRED REVIEWS Require pull request reviews before changes are merged.		SIGNED COMMITS Ensure commit/signature verification for integrity and accountability.		AUDITABILITY Every change is tracked, reviewable, and linked to version control history.
---	---	---	---	---	--	---	---	---	--



ENTERPRISE ADVICE

Production promotion should produce a reviewable Git change, not an invisible mutation inside the cluster.



INVISIBLE MUTATION
kubectl apply ...
Manual change
UI update

VS



REVIEWABLE GIT CHANGE
Pull Request
Code Review
Automated Checks
Audit Trail

8. HELM WITH ARGO CD

VERIQTA

1. HELM INTEGRATION

	Argo CD integrates with Helm to generate Kubernetes manifests from charts.
	Argo CD renders the chart using the provided values.
	Argo CD compares and synchronizes the resulting manifests to the Kubernetes cluster.
	Argo CD manages the resources. Helm is used for rendering, not for lifecycle.

2. HELM SOURCE OPTIONS

SOURCE TYPE	DESCRIPTION
 Helm Repositories	Use charts from Helm repositories (e.g., bitnami, nginx).
 OCI Sources	Pull charts from OCI registries (e.g., oci://ghcr.io/org/charts/app).
 Chart Versions	Pin to specific chart versions for reproducibility.
 Values Files	Provide custom values via files stored in Git.
 Inline Values	Define values directly in the Application manifest.

VALUE PRECEDENCE (HIGH → LOW)

- Parameters (--set / --set-string)
- Inline values (values)
- External values files (valuesFiles)
- Chart default values (values.yaml)

OTHER HELM SETTINGS

- Release Name: Set explicitly or use default (Application name).
- Namespace: Defined in destination.
- Skip CRDs: Enable if CRDs are managed separately.

3. PRODUCTION PATTERNS

PATTERN	DESCRIPTION	BEST FOR
 Shared Chart + Env Values	One chart used by all environments. Each environment has its own values file or directory.	Most teams and standard workloads
 External Values Repositories	Values are stored in separate Git repository and referenced by Argo CD.	Large orgs, separation of duties, reuse
 Multiple Sources	Chart from one repo and values from another using Argo CD multiple sources.	Complex setups, reusability
 Version Pinning	Pin chart version and appVersion for stable and repeatable deployments.	Production stability and auditability



QUICK REMINDERS

- ✓ Define releaseName if different from Application name.
- ✓ Always pin chart versions.
- ✓ Keep values files in Git.
- ✓ Use multiple sources for clean separation.
- ✓ Avoid using UI to manage Helm values in production.

4. HELM EXAMPLE – APPLICATION SPEC

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://charts.bitnami.com/bitnami # Helm repo
    chart: nginx # Chart name
    targetRevision: 15.3.2 # Chart version
    helm:
      valueFiles:
        - values-common.yaml # Repo path
        - values-prod.yaml # Repo path
      parameters:
        - name: replicaCount
          value: "3" # --set replicaCount=3
        - name: image.tag
          value: "1.25.3" # --set image.tag=1.25.3
  destination:
    server: https://kubernetes.default.svc # Cluster API
    namespace: my-app # K8s namespace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - PruneLast=true
```

5. TROUBLESHOOTING HELM WITH ARGO CD

 RENDERING FAILURES - Syntax errors in values - Missing template values - Invalid Helm functions - Run: <code>argocd app get APP --o yaml</code> - Check events tab	 MISSING VALUES - Wrong values file path - Key not found in values - Incorrect indentation - Validate with: <code>helm template . -f values.yaml</code>	 DEPENDENCY FAILURES - Missing chart dependencies - Network or repo issues - Run: <code>helm dependency build</code> - Or helm dep up	 UNEXPECTED MANIFESTS - Wrong values overriding - Chart default surprises - Use <code>argocd app manifests APP</code> - Compare rendered output	 CHART VERSION MISMATCH - Chart not updated - Deprecated APIs in chart - Pin compatible versions - Verify targetRevision
---	--	--	---	---



IMPORTANT CONCEPT

Argo CD uses Helm primarily for manifest generation. Argo CD manages the resulting Kubernetes resources.

Helm Chart + Values



Kubernetes Manifests (YAML)



Argo CD (Sync & Manage)

9. KUSTOMIZE AND DIRECTORY-BASED APPLICATIONS

1. KUSTOMIZE CORE CONCEPTS

	BASE	Canonical, reusable Kubernetes manifests that represent the common configuration.
	OVERLAYS	Environment-specific customizations built on top of one or more bases.
	PATCHES	Strategic Merge Patches or JSON 6902 patches to modify resources.
	IMAGES	Update container images, tags, pull policy, and other image fields.
	COMPONENTS	Reusable configuration fragments (e.g., logging, monitoring, sidecars).
	NAME PREFIXES & SUFFIXES	Add prefixes or suffixes to resource names.
	LABELS & ANNOTATIONS	Add or update labels and annotations consistently.

2. DIRECTORY APPLICATIONS

FEATURE	DESCRIPTION
 PLAIN YAML	Any valid Kubernetes YAML manifest can be used.
 RECURSIVE DISCOVERY	Argo CD discovers manifests recursively from the configured path.
 INCLUDE PATTERNS	Use include patterns to select specific files or directories.
 EXCLUDE PATTERNS	Use exclude patterns to ignore files or directories.
 JSON MANIFESTS	JSON format manifests are supported alongside YAML.

EXAMPLE ARGO CD APPLICATION (DIRECTORY)

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/my-app-config.git
    targetRevision: main
    path: overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

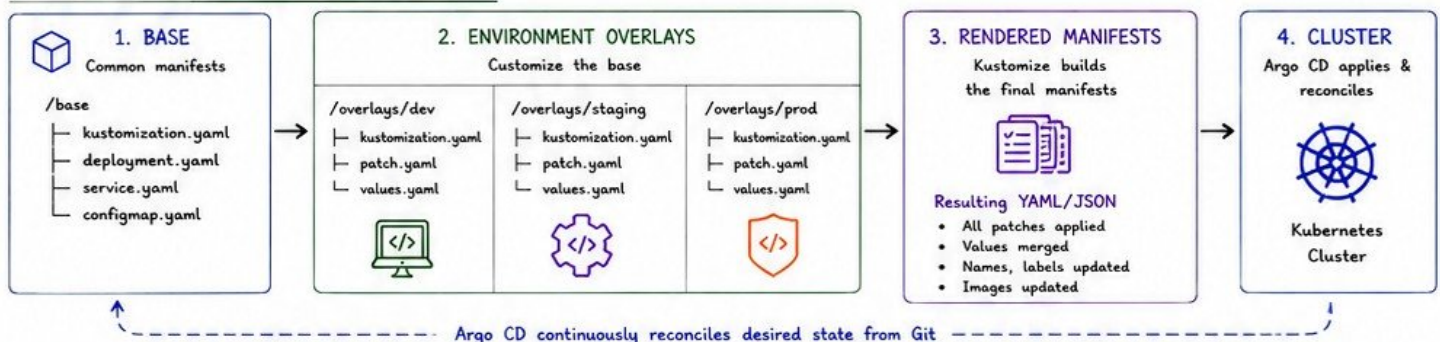
```

Git repository
Branch or tag
Path to overlay
Cluster API
Target namespace
Auto sync
Prune resources
Self-heal drift

3. PRODUCTION ARCHITECTURE WITH KUSTOMIZE

LAYER	PURPOSE	EXAMPLE PATH
 BASE	Common resources shared across all environments.	/base
 DEVELOPMENT OVERLAY	Development-specific customizations.	/overlays/dev
 STAGING OVERLAY	Staging-specific customizations.	/overlays/staging
 PRODUCTION OVERLAY	Production-specific customizations.	/overlays/prod

4. KUSTOMIZE FLOW: BASE TO CLUSTER



PRODUCTION NOTE

- Keep environment differences explicit and easy to review.
- Prefer fewer, larger patches over many small layered patches.
- Avoid deep patch chains that make the final state hard to understand.
- Structure repositories for clarity, scalability, and team ownership.

GOOD



- Base + simple overlay
- Clear patches
- Minimal overrides
- Easy to review

AVOID



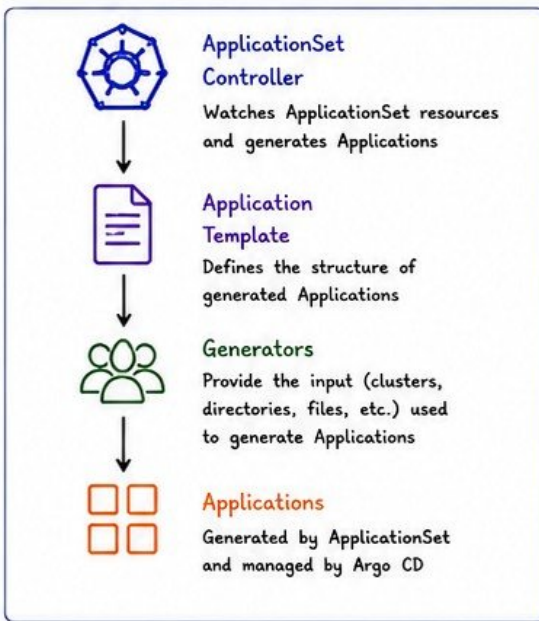
- Many overlay layers
- Complex patch chains
- Hidden implicit changes
- Hard to debug

10. APPLICATIONSET FOR FLEET-SCALE GITOPS

VERIQTA

VERIQTA

1. APPLICATIONSET ARCHITECTURE



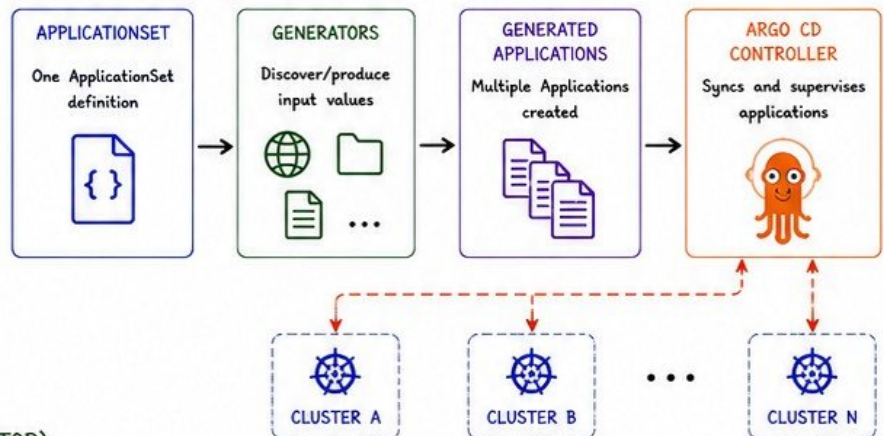
2. GENERATORS

GENERATOR	DESCRIPTION	BEST FOR
List	Static list of values defined inline in the ApplicationSet.	Simple, known set of values (e.g., envs).
Cluster	Discovers clusters registered in Argo CD.	Deploy to all or selected clusters.
Git Directory	Scans directories in a Git repo and generates per directory.	Env/tenant directory structures.
Git File	Reads structured data (YAML/JSON) from a file in Git.	Centralized config or inventory files.
Matrix	Combines two or more generators (cartesian product).	Multi-dimension combos (e.g., cluster x env).
Merge	Merges parameters from multiple generators.	Combine and enrich data from sources.
SCM Provider	Discovers repositories via SCM provider APIs.	Repo onboarding and multi-repo setups.
Pull Request	Creates Applications for each pull request.	Preview environments per PR.
Cluster Decision Resource	Uses Argo CD Cluster Decision Resource for dynamic selection.	Policy-based cluster selection.

3. PRODUCTION USE CASES

Multi-Cluster Deployment	Deploy the same application across many clusters consistently.
Environment Generation	Generate dev, staging, prod, and other environments automatically.
Tenant Onboarding	Onboard new tenants with isolated config and namespaces.
Regional Deployments	Roll out applications to regions with consistent policies.
Cluster Fleet Management	Manage large fleets with labels, tags, and dynamic grouping.

4. HOW APPLICATIONSET WORKS



5. EXAMPLE APPLICATIONSET (CLUSTER GENERATOR)

```

apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: my-apps
  namespace: argocd
spec:
  generators:
    - cluster:
        selector:
          matchLabels:
            env: production
  template:
    metadata:
      name: my-app-{{name}}
    spec:
      project: default
      source:
        repoURL: https://github.com/org/my-app.git
        targetRevision: main
        path: overlays/prod
      destination:
        server: '{{server}}'
        namespace: my-app
      syncPolicy:
        automated:
          prune: true
          selfHeal: true

```

WHAT THIS DOES

- ✓ Discovers all clusters labeled env=production.
- ✓ Creates one Application per matching cluster.
- ✓ Deploys the same app to each cluster.
- ✓ Keeps everything GitOps and synchronized.

TIPS

- Use labels on clusters for grouping and targeting.
- Prefer structured generators over inline lists for scale.
- Combine generators with Matrix for advanced scenarios.
- Test generators locally before applying in production.



SCALING NOTE

ApplicationSet should eliminate repetitive Application definitions without hiding deployment intent.



GOOD

- Clear templates
- Explicit parameters
- Traceable generation
- Reviewable changes



AVOID

- Hidden implicit logic
- Complex generator chains
- Overly dynamic naming
- Unreviewable mutations

VERIQTA

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta

11. APPPROJECTS AND MULTI-TENANT GOVERNANCE

APPPROJECT CONTROLS

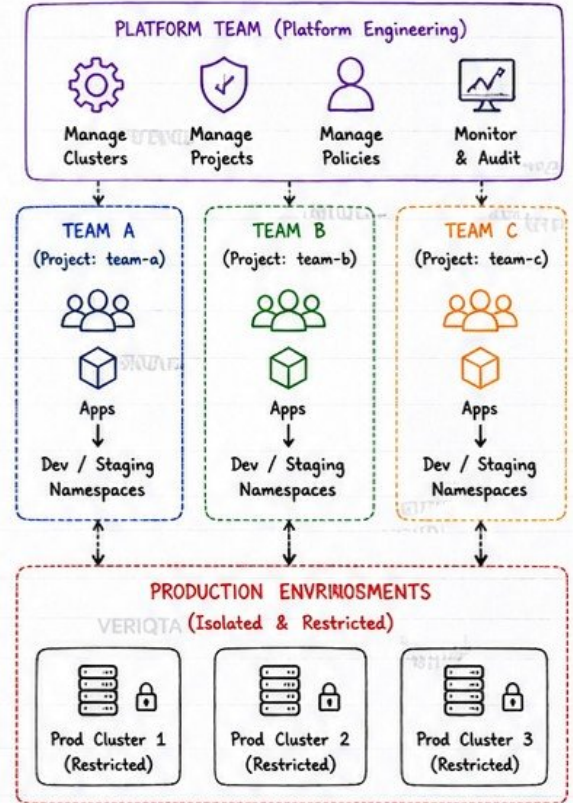
AppProjects define what an application team is ALLOWED to deploy and where.

- Allowed source repositories
Which Git repos can be used
- Allowed destinations
Which clusters & namespaces
- Cluster resources
Cluster-scoped resources that are allowed
- Namespace resources
Namespace-scoped resources that are allowed
- Resource allow lists
Kinds that can be created/managed
- Resource deny lists
Kinds that are explicitly blocked
- Project roles
Define who can access and what they can do
- Orphaned resource monitoring
Detect resources no longer defined in Git

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: team-a-project
  namespace: argocd
spec:
  description: Project for Team A
  sourceRepos:
    - 'https://github.com/org/team-a-*'
    - 'https://github.com/org/shared-charts'
  destinations:
    - namespace: team-a-*
      server: https://kubernetes.default.svc
  clusterResourceWhitelist:
    - group: '*'
      kind: Namespace
  namespaceResourceWhitelist:
    - group: ''
      kind: '*'
  namespaceResourceBlacklist:
    - group: ''
      kind: Secret
  roles:
    - name: team-a-developer
      description: Read & sync own apps
      policies:
        - p, proj:team-a-project:team-a-developer, applications, get, team-a-project/*, allow
      groups:
        - team-a-devs
  orphanedResources:
    warn: true
    ignore:
      - group: ''
        kind: ConfigMap
        name: legacy-config
        namespace: team-a-*
```

MULTI-TENANCY MODEL

AppProjects create safe boundaries between teams and environments.



SECURITY DESIGN PRINCIPLES

- ☒ Restrict destinations
Allow teams to deploy only to specific clusters and namespaces.
- ☒ Restrict repositories
Limit teams to approved repositories only.
- ☒ Restrict privileged resource types
Block cluster-admin level resources (e.g., ClusterRoleBinding, CRDs).
- ☒ Separate sensitive environments
Keep production projects and clusters separate from dev/staging.

WHY THIS MATTERS

- Prevents accidental or malicious deployments outside allowed areas.
- Limits blast radius of a compromised account.
- Enforces least privilege at the GitOps level.
- Improves auditability and compliance.

APPPROJECT PERMISSIONS (EXAMPLE)

ROLE	CAN DO	CANNOT DO
Viewer	View applications, logs, history, diffs, health	Sync, delete, create, modify anything
Developer	Sync applications in allowed namespaces	Create projects, modify RBAC, access other teams' apps
Operator	Sync, delete apps, manage resources in project	Modify AppProject settings, RBAC, cluster access
Admin	Full access within project scope	Access outside project boundaries

SECURITY REMINDER



The "default" project is convenient but provides weak isolation. It allows wide access to clusters, namespaces, and resources. Production environments should use deliberately scoped AppProjects with least-privilege rules.

Never rely on the default project in production.

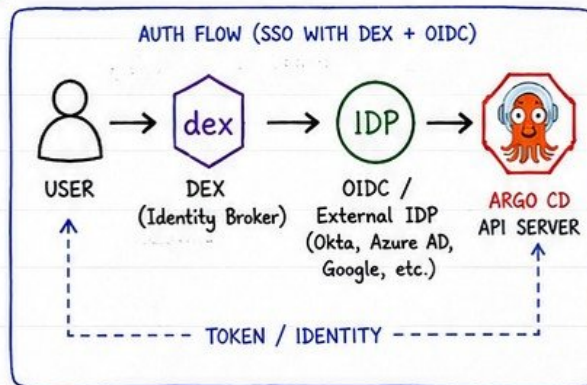
BEST PRACTICES

- ☒ One AppProject per team or application domain.
- ☒ Restrict everything by default, allow only what's needed.
- ☒ Use environments-specific projects (dev, staging, prod).
- ☒ Review AppProject rules regularly.
- ☒ Audit orphaned resources to avoid drift and cost.
- ☒ Document who owns each project and why.

12. AUTHENTICATION, RBAC, SSO, AND AUTHORIZATION

AUTHENTICATION OPTIONS

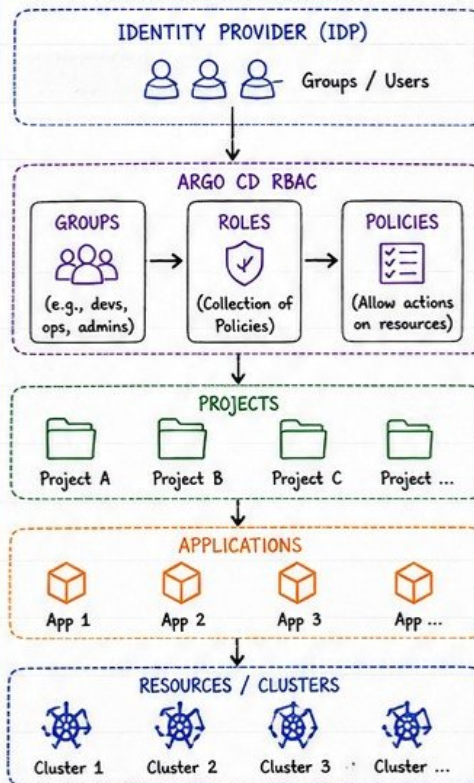
- Local users
Built-in username/password
- SSO (Single Sign-On)
Enterprise identity integration
- Dex
Identity broker for SSO
- OIDC (OpenID Connect)
Standard auth protocol
- External Identity Providers
Okta, Azure AD, Google, GitHub, Keycloak, etc.



PRODUCTION SECURITY PRINCIPLES

- ✓ **Least privilege**
Grant users only the minimum permissions needed to do their job.
- ✓ **Disable unnecessary local accounts**
Use SSO/OIDC for all human users.
- ✓ **Protect administrator access**
Limit admin group membership.
Use break-glass accounts if required.
- ✓ **Short-lived authentication**
Use tokens with short TTLs via OIDC.
Avoid long-lived static credentials.
- ✓ **Group-based authorization**
Map IdP groups to roles. Avoid user-by-user permission management.
- ✓ **Audit access changes**
Enable audit logs. Review role changes and group mappings regularly.

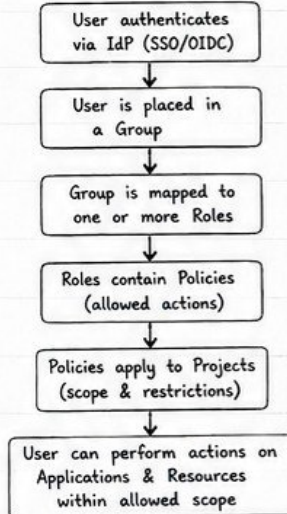
RBAC MODEL OVERVIEW



RBAC BUILDING BLOCKS

- Policies**
Define what actions are allowed on which resources.
- Roles**
Collection of policies.
- Groups**
Identity groups mapped from IdP (e.g., teams, departments).
- Projects**
Isolate applications, clusters, namespaces and restrictions.
- Application Permissions**
Allow actions on applications within a project.
- Resource Actions**
get, list, sync, delete, update, override, approve, etc.

PERMISSIONS FLOW



EXAMPLE PERMISSIONS

ROLE	WHO	CAN DO	CANNOT DO
Read-only Developer	Developers, Viewers	- View applications - View logs & events - View manifests & diff - Sync (if allowed by policy)	- Delete applications - Modify settings - Change RBAC - Manage projects
Deployment Operator	Ops Engineers, Release Managers	- Sync applications - Get, list resources - Restart / rollback (if allowed) - View logs & events	- Manage projects - Change RBAC - Delete applications - Modify Argo CD config
Platform Administrator	Platform Team, SREs, Admins	- Full access to Argo CD - Manage projects - Manage RBAC - Manage clusters & settings	None

SECURITY REMINDER



Argo CD can modify production workloads. Treat access to Argo CD as privileged infrastructure access.

KEY TAKEAWAY



Strong authentication + well-designed RBAC = secure GitOps.
Design for least privilege, isolation, and auditability.
Never rely on default access.

13. SECRETS MANAGEMENT IN GITOPS

THE GITOPS SECRETS PROBLEM



Git is the source of truth—but secrets in Git create serious risks.

- Secrets are sensitive and must be protected from unauthorized access.
- Repositories can be copied, leaked or exposed.



WHY PLAINTEXT SECRETS MUST NOT LIVE IN GIT

- Anyone with repo access can read secrets.
- Git history retains secrets forever.
- Forks and clones can leak secrets.
- CI logs and build artifacts may expose secrets.
- Human error can commit secrets.

ENCRYPTION AT REST VS EXPOSURE DURING RECONCILIATION



Encryption at Rest

- Protects secrets stored in Git or secret stores.
- Examples: SOPS, KMS, Vault encryption, cloud KMS.



Exposure During Reconciliation

- Secrets must be decrypted to be used by workloads.
- Decrypted secrets exist in memory, on nodes, and in network.
- Limit exposure window and scope.

COMMON APPROACHES

EXTERNAL SECRETS OPERATOR



- Pull secrets from external stores into Kubernetes.
- Supports many secret stores.
- Keeps secrets out of Git.

HASHICORP VAULT INTEGRATIONS



- Dynamic secrets.
- Fine-grained access control.
- Strong audit and security model.

SEALED SECRETS



- Encrypt secrets with cluster public key.
- Only that cluster can decrypt.
- Safe to store in Git.

SOPS-BASED WORKFLOWS



- Encrypt secrets using GPG, KMS, or Age.
- Decrypt at runtime in CI/CD or controllers.
- Human-readable YAML.

CLOUD SECRET MANAGERS



- AWS Secrets Manager
- Azure Key Vault
- Google Secret Manager
- Managed, scalable, highly available.

SECRET-MANAGEMENT PLUGINS



- Argo CD plugins or custom tools.
- Extend Argo CD to integrate with secret systems.

SECURITY CONSIDERATIONS



Repository Compromise
Someone gaining repo access can steal encrypted or reference configs.



Decryption Keys
Protect keys (KMS, GPG, Vault tokens).
Compromised keys = total breach.



Service Account Permissions
Use least-privilege IAM / Kubernetes permissions for secret access.



Secret Rotation
Rotate secrets regularly to limit blast radius of leaked credentials.

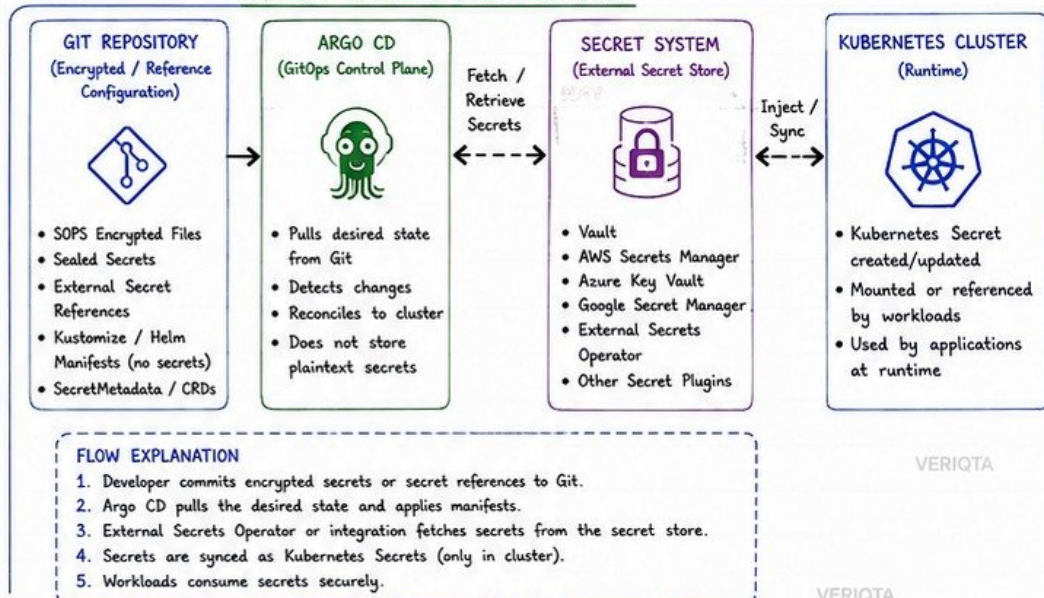


Audit Trails
Enable logging and auditing on secret stores and access events.



Credential Leakage
Avoid exposing secrets in logs, manifests, CI output, or error messages.

GITOPS SECRETS ARCHITECTURE



ENTERPRISE ADVICE



Git should describe **HOW** workloads obtain secrets, not store the secrets themselves.
Use Git for configuration, references, and encryption—not for plaintext secrets.
Treat secret systems as critical production infrastructure with strong access controls and monitoring.



YouTube: veriqta



GitHub: veriqta



LinkedIn: veriqta



X: veriqta



Instagram: veriqta



Telegram: veriqta

14. MULTI-CLUSTER ARGO CD ARCHITECTURE

CLUSTER REGISTRATION



In-cluster Deployment

- Argo CD runs inside the same cluster it manages.



External Clusters

- Argo CD manages clusters outside its own Kubernetes environment.



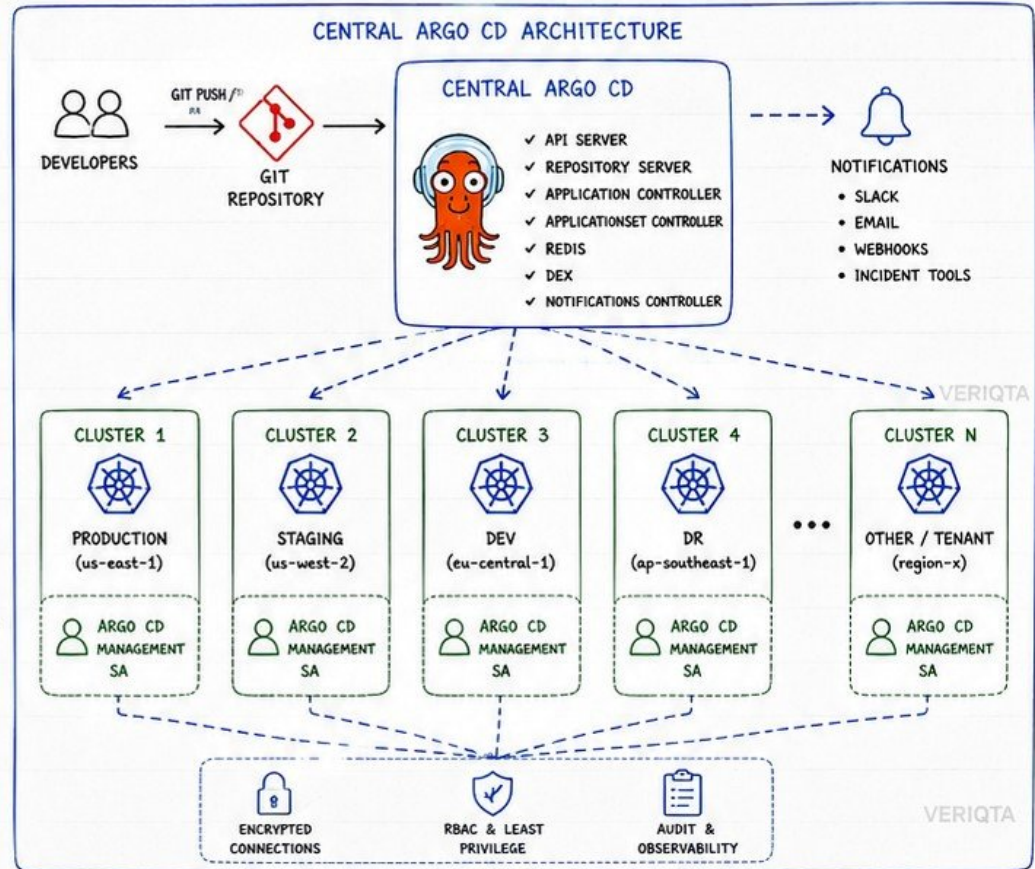
Cluster Credentials

- Stored as Kubernetes Secrets.
- Contain API server endpoint, CA certificate, and token.



Service Accounts

- Dedicated service accounts on each managed cluster.
- Grant minimum required permissions for Argo CD to work.



ARCHITECTURAL CHOICES



Central Control Plane

- One Argo CD instance manages many clusters.
- Simpler to operate.
- Larger blast radius.



Regional Argo CD

- One Argo CD per region.
- Improves latency and resilience.
- More operational overhead.



Per-cluster Argo CD

- Argo CD installed inside each cluster.
- Small blast radius and full isolation.
- Harder to manage at scale.



Environment-isolated Argo CD

- Separate Argo CD for dev, staging, prod.
- Strong isolation and compliance.
- Higher cost and complexity.

EVALUATION CRITERIA



Blast Radius

How many clusters are affected by a failure or misconfiguration.



Network Connectivity

Latency, bandwidth, and firewall considerations between Argo CD and target clusters.



Credentials

How credentials are stored, rotated, and secured.



Availability

High availability requirements for the control plane.



Compliance Boundaries

Data residency, legal boundaries, and security zones.



Cluster Count

Number of clusters today and expected growth.



Operational Ownership

Who owns Argo CD? Platform team, SRE team, or decentralized teams?

FAILURE SCENARIOS



Target Cluster Unreachable

Network issues, API server down, or firewall blocks.



Expired Credentials

Service account token expired or certificate rotated.



API Server Latency

High latency causes slow reconciliation and timeouts.



Central Argo CD Outage

Control plane down affects all managed clusters.



KEY TAKEAWAYS

- ✓ Choose architecture based on risk, scale, and organization boundaries.
- ✓ Secure credentials and isolate privileged access.
- ✓ Monitor connectivity and API health continuously.
- ✓ Design for failure. Test recovery regularly.



PRODUCTION NOTE

A central Argo CD control plane simplifies GitOps at scale but becomes a critical dependency. Harden it, monitor it, back it up, and plan for disaster recovery.

Always apply least privilege to cluster credentials and service accounts.



YouTube: veriqta



GitHub: veriqta



LinkedIn: veriqta



X: veriqta



Instagram: veriqta



Telegram: veriqta

15. PROGRESSIVE DELIVERY AND ARGO ROLLOUTS

ARGO CD vs ARGO ROLLOUTS

Understand the separation between GitOps reconciliation and advanced deployment strategies.

ARGO CD (GITOPS)



- Git is the source of truth
- Declarative state
- Syncs resources to cluster
- Reconciliation & drift correction
- Ensures desired state

ARGO ROLLOUTS



- Advanced deployment strategies
- Traffic management
- Analysis & measurements
- Progressive delivery
- Safe promotions & rollbacks

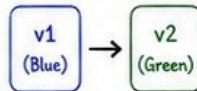


Argo CD deploys the desired resources.

Argo Rollouts controls how traffic moves between versions.

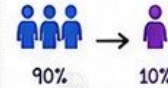
PROGRESSIVE DELIVERY STRATEGIES

BLUE-GREEN



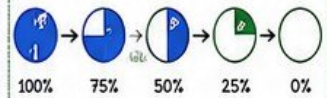
Switch traffic all at once.

CANARY



Gradually shift traffic to new version.

TRAFFIC SHIFTING



Incrementally move traffic between versions.

ANALYSIS



Measure metrics & run automated analysis.

PROMOTION



Promote to stable after successful analysis.

ROLLBACK



Rollback quickly on failure or regression.

ROLLOUTS CONCEPTS

ROLLOUT



A custom resource that defines the desired state and strategy for delivering an application.

REPLICASET



Creates and manages ReplicaSets for each revision during rollout.

ANALYSISTEMPLATE



Defines metrics, success criteria, and failure conditions.

ANALYSISRUN



Executes an AnalysisTemplate and provides real-time results.

EXPERIMENT



Test changes in a controlled environment before promoting.

INTEGRATIONS

INGRESS CONTROLLERS



- NGINX Ingress
- Traefik
- HAProxy
- ALB Ingress Controller

Manage traffic routing and canary/blue-green switching.

SERVICE MESHES



- Istio
- Linkerd
- Consul

Advanced traffic management, mTLS, retries, and observability.

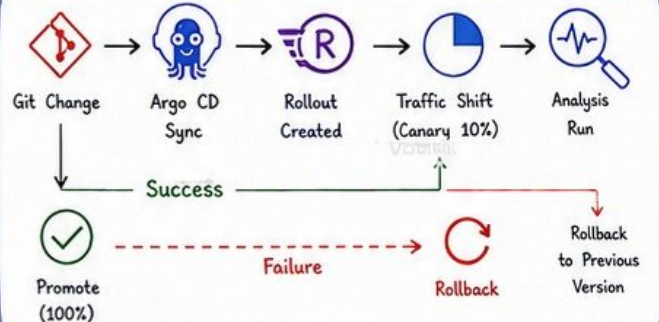
METRICS PROVIDERS



- Prometheus
- Datadog
- Grafana
- Wavefront

Collect metrics for analysis, alerts, and decisions.

ROLLOUT FLOW (EXAMPLE: CANARY)



HOW IT WORKS TOGETHER

1. Developer commits change → Git.
2. Argo CD syncs manifests to the cluster.
3. Argo Rollouts creates/updates the Rollout resource.
4. Rollouts controller executes the strategy (canary, blue-green, etc.).
5. Traffic is shifted gradually while analysis runs.
6. If successful → promote. If failed → rollback.

PRODUCTION FAILURE



A successful Git synchronization does not prove that a new application version is healthy. Always validate with real user traffic and automated analysis before full rollout.



YouTube: veriqta



GitHub: veriqta



LinkedIn: veriqta



X: veriqta



Instagram: veriqta



Telegram: veriqta

16. NOTIFICATIONS, WEBHOOKS, AND OPERATIONAL INTEGRATION

ARGO CD NOTIFICATIONS



Argo CD Notifications is a separate controller that watches events and sends alerts through integrations.



Triggers

Define when notifications should fire.



Templates

Define message content using Go templates.



Subscriptions

Map events to destinations (integrations).



Application Events

Events from applications and Argo CD.

IMPORTANT EVENTS



Sync Succeeded

Application synced successfully.



Sync Failed

Application sync failed.



Application Degraded

Application health is degraded.



Health Changed

Application health status changed.



Other Events

Created, deleted, updated, etc.

INTEGRATIONS

Send notifications to the tools your team already uses.



Slack

Channels, threads, rich messages.



Email

SMTP, SendGrid, SES, etc.



Webhooks

Send HTTP POST payloads.



Incident-Management Systems

PagerDuty, Opsgenie, Jira Service Management, etc.



GIT WEBHOOKS



Faster Refresh

Immediate updates when Git changes occur.



Repository Events

Push, PR, tag, release, delete, comment, etc.



Webhook Security

Use secrets, HMAC signatures, and HTTPS to secure webhook endpoints.

END-TO-END NOTIFICATION FLOW

GIT REPOSITORY



Push / PR / Tag / Delete



Webhook Event

ARGO CD



- ✓ Detects Change
- ✓ Reconciles
- ✓ Generates Events

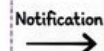


Event

NOTIFICATIONS CONTROLLER



- ✓ Evaluates Triggers
- ✓ Renders Templates
- ✓ Executes Subscriptions



Notification

INTEGRATIONS



Delivers Alerts

OPERATIONAL DESIGN PRINCIPLES



Actionable Alerts

Alert only on conditions that require human attention and action.



Environment Context

Include environment, cluster, namespace, and region in every notification.



Application Ownership

Route alerts to the correct team or service owner.



Avoid Notification Fatigue

Too many noisy alerts cause teams to ignore important ones.

EXAMPLE NOTIFICATION (SLACK)



Argo CD APP 10:30 AM

✓:white_check_mark Sync Succeeded

Application : payments-api
Environment : production
Cluster : prod-us-east-1
Namespace : payments
Revision : 8f3a2c1
Deployed By : github-actions

[View in Argo CD](#)

[View Commit](#)

EXAMPLE WEBHOOK PAYLOAD

```
{
  "event": "sync.succeeded",
  "app": "payments-api",
  "project": "prod-apps",
  "namespace": "payments",
  "cluster": "prod-us-east-1",
  "revision": "8f3a2c1",
  "status": "Synced",
  "health": "Healthy",
  "message": "application synced successfully"
}
```



PRODUCTION NOTE

Alert on conditions requiring human action, not every reconciliation event.



Human Action Required



Degraded Health



Repeated Failures



Ignore Healthy Syncs



Ignore Normal Diffs



YouTube: veriqta



GitHub: veriqta



LinkedIn: veriqta



X: veriqta



Instagram: veriqta



Telegram: veriqta

17. OBSERVABILITY, METRICS, LOGS, AND PERFORMANCE

OBSERVABILITY PILLARS

Good observability helps you understand what Argo CD is doing, how healthy it is, and why something failed.



METRICS

Quantitative measurements of system behavior.



LOGS

Detailed events and runtime information.



EVENTS

Kubernetes and Argo CD events and state changes.



TRACES (Optional)

Request flow across components.

ARGO CD METRICS: WHAT TO MONITOR

COMPONENT	KEY METRICS EXAMPLES	WHAT IT TELLS YOU
 CONTROLLER METRICS	- argocd_app_reconcile_total - argocd_app_reconcile_duration_seconds - argocd_app_sync_total - workqueue_depth	Reconciliation behavior, sync performance, and controller workload.
 API SERVER METRICS	- http_request_duration_seconds - grpc_requests_total - active_connections - api_request_total	API responsiveness, traffic volume, and error rates.
 REPOSITORY SERVER METRICS	- repo_server_request_total - repo_server_request_duration_seconds - manifest_generation_duration_seconds - cache_hit_ratio	Git access latency, manifest rendering performance, cache effectiveness.
 APPLICATIONSET METRICS	- applicationset_reconcile_total - applicationset_generation_total - applicationset_controller_queue_depth	ApplicationSet controller health and queue pressure.
 KUBERNETES EVENTS	- Warning / Error events - FailedCreate / FailedSync - Degraded / Unhealthy	Cluster-level issues impacting applications and resources.

WHAT TO MONITOR (OPERATIONAL VIEW)

 RECONCILIATION DURATION Detect slow or stuck reconciliations. High values = controller or cluster issues.	 KUBERNETES API LATENCY High API latency slows reconciliation and status updates.
 SYNC FAILURES Track failure rate and reasons. Spike = configuration, cluster, or permission problem.	 CONTROLLER QUEUE BEHAVIOR Monitor queue depth, retries, and rate of processing.
 APPLICATION HEALTH Healthy, Degraded, Progressing, Missing. Measure over time.	 REPOSITORY RENDERING PERFORMANCE Manifest generation time directly affects scale and sync speed.
 GIT REQUEST LATENCY Slow Git responses impact sync and manifest generation.	

COMPONENT LOGS: WHAT TO LOOK FOR

COMPONENT	IMPORTANT LOG SIGNALS
 CONTROLLER	- Reconciliation errors - Permission denied - Resource apply failures - Diff calculation errors
 API SERVER	- gRPC errors - Authentication failures - Rate limiting - Internal server errors
 REPOSITORY SERVER	- Git fetch failures - Render errors - Helm / Kustomize errors - Timeout errors
 APPLICATIONSET CONTROLLER	- Template rendering errors - Generator failures - Sync policy issues

PERFORMANCE CONSIDERATIONS AT SCALE

LARGE MONOREPOSITORIES  - Many changes trigger many apps - Use path scoping and AppProject restrictions - Consider repo segmentation	THOUSANDS OF APPLICATIONS  - Controller memory and CPU increase - Use ApplicationSets and generators - Monitor queue depth and sync time	MANIFEST GENERATION COST  - Complex Helm/Kustomize takes more time - Optimize templates - Use caching and diff customizations	CONTROLLER SHARDING  - Split controllers by AppProject - Reduces queue contention - Improves isolation	REPOSITORY SERVER SCALING  - Horizontal scaling improves render throughput - Enable caching - Monitor CPU & memory usage	KUBERNETES API PRESSURE  - High API QPS causes throttling - Tune QPS/Burst - Use local clusters or regional setups	CACHING  - Repo cache reduces Git load - Manifest cache improves diff performance - Tune cache TTLs
---	---	--	---	---	---	--

OBSERVABILITY STACK (EXAMPLE)



PERFORMANCE TIP

Scaling Argo CD requires understanding which component is saturated. Adding replicas everywhere can increase cost without removing the actual bottleneck. Measure → Identify → Optimize → Scale.

18. TROUBLESHOOTING ARGO CD IN PRODUCTION

TROUBLESHOOTING WORKFLOW

- 1 Check Application sync status.
- 2 Check Application health.
- 3 Inspect resource differences.
- 4 Inspect Application conditions.
- 5 Review controller logs.
- 6 Review Repository Server logs.
- 7 Validate repository access.
- 8 Validate destination cluster access.
- 9 Render manifests independently.
- 10 Inspect Kubernetes events.

COMMON FAILURES AND THEIR MEANING

OutOfSync	- Live state differs from desired Git state. - Manual changes, missing resources, or sync not run.
Unknown	- Argo CD cannot determine application health. - Cluster unreachable or resource info unavailable.
Degraded	- Resources are unhealthy. - Pods failing, probes failing, dependencies down.
Repository Authentication Failure	- Invalid credentials, expired token, or lack of access. - Check repo URL, credentials, and network.
Manifest Generation Failure	- Kustomize, Helm, or plugin failed to render. - Invalid values, missing charts, or template errors.
Comparison Error	- Argo CD failed to compare live vs desired. - Large objects, invalid resource, or API limitations.
Sync Failure	- Sync started but failed on one or more resources. - Validate logs, events, permissions, and quotas.
Permission Denied	- Insufficient RBAC on cluster or namespace. - Service account lacks required permissions.
Resource Stuck Terminating	- Finalizers blocking deletion. - CRD deletion, dangling finalizers, or dependencies.
Cluster Unreachable	API server unreachable, network issue, or expired credentials.
Unexpected Drift	- External controller or operator changed resources. - Investigate controllers and automation.

USEFUL ARGO CD COMMANDS

`argocd app get <app>`
Show app status, health, sync, and summary.

`argocd app diff <app>`
Show differences between desired and live state.

`argocd app sync <app>`
Synchronize the application to the desired state.

`argocd app history <app>`
Show revision and sync history.

`argocd app manifests <app>`
Render and show manifests from the repository.

`argocd app resources <app>`
List all managed Kubernetes resources.

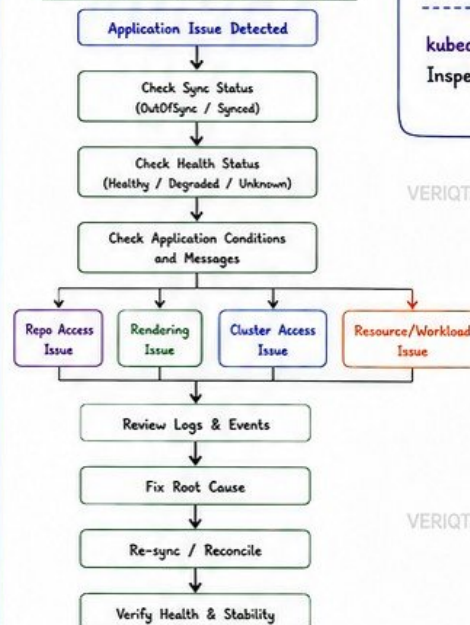
`kubectl describe <resource>`
Detailed information about a resource.

`kubectl logs <pod> -n <ns>`
Inspect pod or controller logs.

TROUBLESHOOTING AREAS AND WHAT TO CHECK

AREA	WHAT TO CHECK	TOOLS / COMMANDS
1. GIT / REPOSITORY	- Repository reachable - Correct revision - Credentials valid	<pre>argocd repo list argocd repo get <repo> git ls-remote <repo></pre>
2. RENDERING / MANIFESTS	- Helm/Kustomize errors - Values and templates - Plugin execution	<pre>argocd app manifests <app> argocd app diff <app> helm template ... kustomize build ...</pre>
3. ARGO CD COMPONENTS	- Controller health - Repo server health - API server connectivity	<pre>kubectl logs deploy/argocd-application-controller -n argocd kubectl logs deploy/argocd-repo-server -n argocd</pre>
4. DESTINATION CLUSTER	- API server reachable - Credentials valid - RBAC permissions	<pre>kubectl cluster-info kubectl auth can-i --list kubectl get nodes</pre>
5. WORKLOAD / RESOURCES	- Kubernetes events - Pod health & status - Probes and dependencies	<pre>kubectl get events -n <ns> kubectl describe <resource> kubectl get pods -n <ns></pre>

QUICK DIAGNOSTIC FLOWCHART



Focus on root cause, not symptoms.



Use logs, events, and conditions.



Validate access before syncing again.



Fix the cause, then verify convergence.

DEBUGGING INSIGHT

Never start production troubleshooting by clicking Sync repeatedly. Identify whether the failure belongs to Git, rendering, Argo CD, Kubernetes, or the workload.



GIT



RENDERING



ARGO CD



KUBERNETES



WORKLOAD



YouTube: veriqta



GitHub: veriqta



LinkedIn: veriqta



X: veriqta



Instagram: veriqta



Telegram: veriqta

19. DISASTER RECOVERY, UPGRADES, SECURITY, AND RELIABILITY

1. DISASTER RECOVERY

- Declarative Configuration**
 - Git is the source of truth.
 - Desired state is always recoverable.
- Repository Recovery**
 - Restore Git repositories from backup or remote.
 - Validate commits and branches.
- Cluster Credentials**
 - Store securely (Secrets / SealedSecrets / Vault).
 - Recreate service accounts if needed.
- Argo CD Configuration Backup**
 - Back up Argo CD settings, applications, projects, AppSets, and RBAC.
 - Backup Redis (if used externally).
- Restore Procedures**
 - Reinstall Argo CD.
 - Restore configuration.
 - Recreate credentials.
 - Sync applications.
- Recovery Testing**
 - Test restore regularly.
 - Validate applications, clusters, and integrations.

Goal: Reduce recovery time and uncertainty.

2. UPGRADE STRATEGY

- Read Release Notes**
 - Review all changes, deprecations, and known issues.
- Check Breaking Changes**
 - Look for API, CRD, or behavior changes.
- CRD Considerations**
 - Backup CRDs.
 - Understand conversion requirements.
- Stage Upgrades**
 - Upgrade in a non-prod environment first.
 - Validate functionality.
- Validate Integrations**
 - Test Git, clusters, SSO, notifications, and plugins.
- Test Rollback Strategy**
 - Know how to roll back.
 - Test rollback in staging.

Upgrade safely. Validate everything.

3. SECURITY HARDENING

- TLS**
 - Use TLS everywhere.
 - Use valid certificates.
- NetworkPolicies**
 - Restrict traffic to only required namespaces and endpoints.
- RBAC**
 - Use least privilege.
 - Restrict admin access.
- SSO**
 - Enforce SSO (OIDC).
 - Disable local accounts when possible.
- Repository Credentials**
 - Use credential templates or external secrets.
 - Rotate credentials regularly.
- Secret Protection**
 - Encrypt/seal secrets.
 - Avoid plaintext secrets in Git.
- Image / Version Pinning**
 - Pin container images by digest or tag.
 - Avoid "latest".
- Restricted Administrative Access**
 - Limit who can access Argo CD admin.
 - Use just-in-time access where possible.

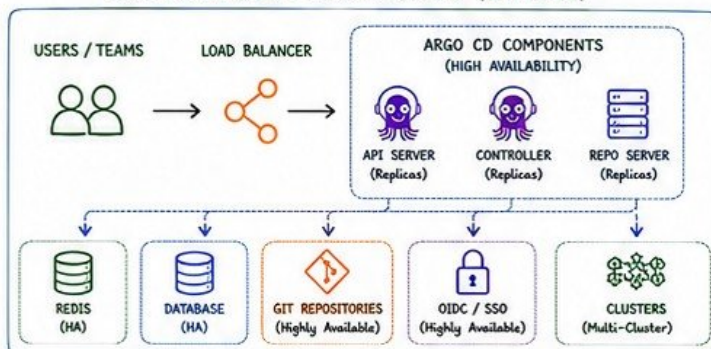
Principle: Least privilege, zero trust.

4. RELIABILITY

- HA Architecture**
 - Run Argo CD in HA mode.
 - Multiple replicas of controller, repo-server, API server.
- Pod Disruption Protection**
 - Use PDBs for all critical components.
- Failure Domains**
 - Distribute across availability zones or regions.
- Dependency Availability**
 - Ensure Redis, DB, SSO, and other dependencies are highly available.
- Git Provider Availability**
 - Use resilient Git hosting.
 - Monitor status and rate limits.

Design for failure. Test for resilience.

HIGH AVAILABILITY ARCHITECTURE (EXAMPLE)



DISASTER RECOVERY CHECKLIST

- ☒ Git repositories are backed up and accessible.
- ☒ Cluster credentials are backed up and restorable.
- ☒ Argo CD configuration is backed up (apps, projects, RBAC).
- ☒ Secrets are encrypted/sealed and recoverable.
- ☒ Documented restore procedures are available.
- ☒ Recovery has been tested in a non-production environment.
- ☒ Team knows responsibilities during a disaster.
- ☒ Monitoring and alerts are restored after recovery.

REAL WORLD LESSON



Git provides **recovery** for desired state, but Git alone does not back up every operational dependency required to restore the Argo CD platform.



KEY TAKEAWAYS

- Git is your source of truth, but not your only dependency.
- Secure, restrict, and audit all access to Argo CD.
- Test upgrades and recovery before you need them.
- Design for failure and validate resilience regularly.
- Reliability is built through architecture, process, and people.



YouTube: veriqta



GitHub: veriqta



LinkedIn: veriqta



X: veriqta



Instagram: veriqta

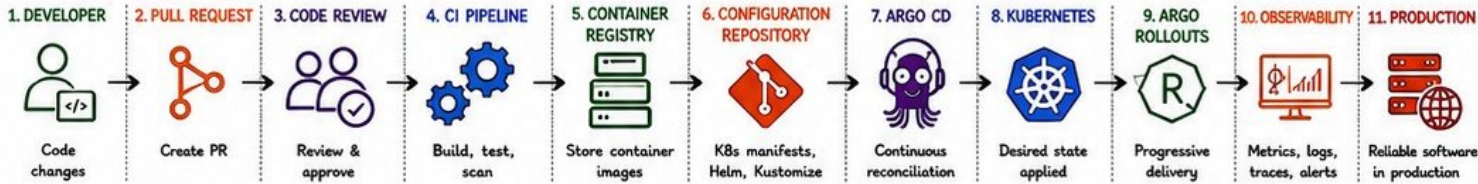


Telegram: veriqta

20. ENTERPRISE ARGO CD ARCHITECTURE AND PRODUCTION READINESS

VERIQTA

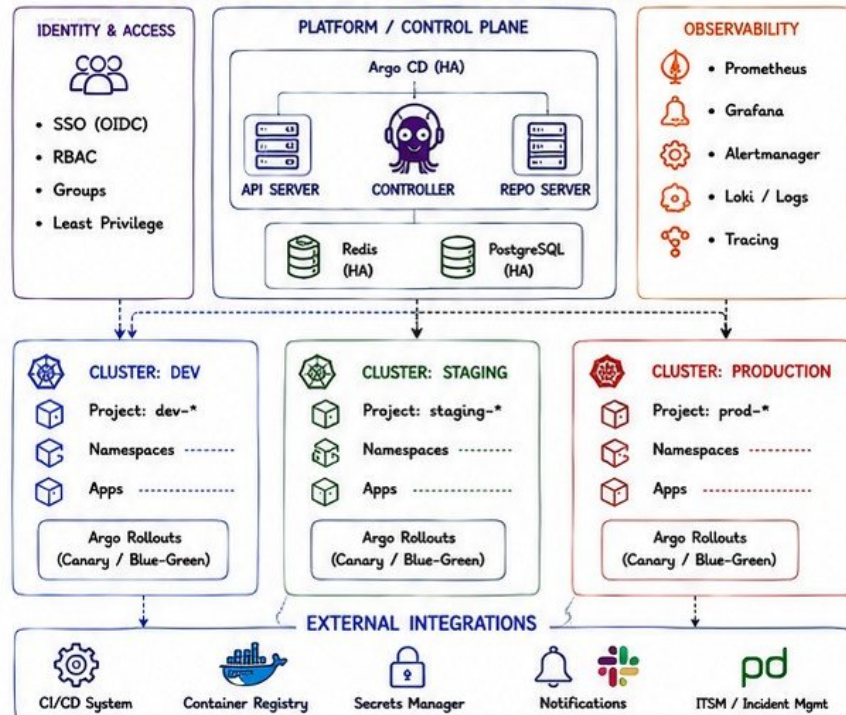
END-TO-END ENTERPRISE GITOPS ARCHITECTURE



ENTERPRISE PRODUCTION CHECKLIST

1	Git repository strategy defined	11	Metrics collected
2	Environment promotion controlled	12	Alerts configured
3	AppProjects configured	13	Repository Server capacity reviewed
4	RBAC follows least privilege	14	Disaster recovery tested
5	SSO enabled	15	Upgrade procedure documented
6	Secrets excluded from plaintext Git	16	Audit trail available
7	Automated pruning intentionally configured	17	Rollback procedure tested
8	Sync behavior documented	18	Security policies reviewed and enforced
9	Multi-cluster boundaries established	19	Operational ownership clearly defined
10	HA configured where required	20	Runbooks and SOPs up to date

ENTERPRISE ARGO CD REFERENCE ARCHITECTURE



ENTERPRISE ARCHITECTURE REVIEW AREAS

SECURITY Access control, network isolation, secrets, encryption, and compliance.	BLAST RADIUS Project scoping, cluster boundaries, and environment isolation.
RELIABILITY High availability, backups, failure domains, and recovery capability.	OBSERVABILITY Metrics, logs, events, traces, and alerting.
SCALABILITY Controller throughput, repo-server scaling, cluster scaling, and performance.	RECOVERY Backups, restore procedures, RTO/RPO, and testing.
GIT GOVERNANCE Branching model, code review, protections, approvals, and audit.	OPERATIONAL OWNERSHIP Defined owners, runbooks, SOPs, and escalation paths.
MULTI-TENANCY AppProjects, namespace boundaries, tenant isolation, and quotas.	

MULTI-CLUSTER BOUNDARIES AND TENANT ISOLATION

ISOLATION LEVELS	TENANT A	TENANT B	TENANT C
PROJECT (Scoping)	AppProject A-*	AppProject B-*	AppProject C-*
REPOSITORIES (Restricted)	Allowed Repos	Allowed Repos	Allowed Repos
DESTINATIONS (Restricted)			
NAMESPACES (Restricted)			
POLICIES (Enforced)	RBAC + SSO + NetworkPolicies + Security Policies	RBAC + SSO + NetworkPolicies + Security Policies	RBAC + SSO + NetworkPolicies + Security Policies



FINAL SENIOR ENGINEER LESSON

Argo CD is not simply a Kubernetes deployment tool. In a mature platform, it becomes part of the production control plane. Design Git permissions, Argo CD privileges, cluster boundaries, reconciliation policies, secrets, and recovery procedures with the same discipline applied to any other critical infrastructure system.



PEOPLE



PROCESS



PLATFORM



RELIABLE SOFTWARE